Parallelizing GPT-2 — "It works on my GPU"

Siggy Sigler, Sam Abderholden, Dawson Matthews

May 5, 2026

1 Paper Summary

Placeholder text.

2 Implementation

Problem Background

In this project, we implement an optimized CUDA version of the forward pass of Andrej Karpathy's *microgpt* [1]. *Microgpt* is a minimal, dependency-free Python implementation of a GPT-style model. It is based on the GPT-2 architecture [2], with a few changes made for simplicity. Both GPT-2 and Karpathy's *microgpt* implement:

- Decoder-only Transformer architecture
- Token + positional embeddings
- Stacked Transformer blocks
- Causal (masked) multi-head self-attention
- Residual connections around attention and MLP sublayers
- Feedforward MLP with expansion and projection
- Autoregressive next-token prediction

Since *microgpt* was intended to showcase the GPT-2 architecture in a single unoptimized Python file, the proposed model has only 4,192 parameters compared to GPT-2's 1.6 billion parameters. In addition to the reduced model size, *microgpt* introduces several simplifications, including the use of RMSNorm instead of LayerNorm, removal of bias terms, and ReLU in place of GELU [1].

In order to keep the scope of our implementation to 10 hours, we focus only on speeding up the forward pass of *microgpt*. Since we believed that comparing a CUDA C++ version to a Python version would introduce speedups due to both our parallelization choices and language differences (C++ vs Python), we first reimplemented *microgpt* in C++, preserving the following:

- Identical layer structure
- Same ordering of operations (norm → attention → residual → norm → MLP → residual)
- Same KV caching mechanism
- Same attention computation (scaled dot-product + softmax)
- Same MLP structure (`fc1` → activation → `fc2`)

Our C++ version is nearly identical, with differences limited to implementation details such as replacing dynamic Python data structures with pre-allocated caches, removing autograd in favor of raw floating-point computation, and expressing all operations as explicit loop-based computations over contiguous memory.

C++ Serial Baseline

Our implementation follows the training setup used in *microgpt*, which operates on a character-level dataset consisting of a set of names $\mathcal{D} = \{s^{(1)}, s^{(2)}, \ldots, s^{(N)}\}$. Each name $s^{(k)} \in \mathcal{D}$ is treated as a sequence of tokens, $s^{(k)} = [t_1, t_2, \ldots, t_T]$, where T denotes the sequence length. The model is trained autoregressively to predict the next token given all previous tokens. Our C++ implementation preserves this setup exactly, performing the same forward-pass computation and next-token prediction task.

Given a tokenized sequence (i.e., a single name) $[t_1, t_2, \ldots, t_T]$, the model computes a sequence of logits $[y_1, y_2, \ldots, y_{T-1}]$, where each y_i represents the model's prediction of token t_{i+1} conditioned on the prefix $[t_1, \ldots, t_i]$. In the baseline implementation, this computation is performed sequentially, iterating over the dataset one name at a time and, within each name, one token at a time, with each position attending over all previous tokens in the sequence. A high level pseudocode of our serial implentation is shown under Algorithm 1.

We denote the embedding dimension as d, the number of Transformer layers as L, and the number of attention heads as H. For each token position i, the model produces a hidden representation $x_i \in \mathbb{R}^d$, which is transformed through attention and MLP operations across all layers.

The computational cost of the forward pass over N sequences is $O(N \cdot L \cdot (C_{\text{attn}} + C_{\text{mlp}}))$, where $C_{\text{attn}} = O(T^2 \cdot d)$ and $C_{\text{mlp}} = O(T \cdot d^2)$. Since the names have approximately constant sequence lengths, we can simplify this to $O(N \cdot L \cdot d^2)$.

Algorithm 1 High-Level Forward Pass

```
loss ← 0
for each sequence s^(k) ∈ D do
    for i = 1 to T − 1 do
        x_i ← TokenEmbedding(t_i) + PositionalEmbedding(i)
        x_i ← RMSNorm(x_i)
        for ℓ = 1 to L do
            x_i ← x_i + Attention(RMSNorm(x_i), {k_j, v_j}_{j≤i})
            x_i ← x_i + MLP(RMSNorm(x_i))
        end for
        y_i ← Linear(x_i)
        loss ← loss − log(Softmax(y_i)[t_{i+1}])
    end for
end for
loss ← loss/total_tokens
```

3 Parallelization Strategy

Using the *perf time* command we were able to get high level timing characteristics of the program, especially how much time our program performing OS overhead versus useful compute work. With nearly zero system time (0.008s out of 2.74s total) and only 269 context switches, the program runs almost entirely in userspace with negligible OS overhead. An IPC of 3.06 with 0.999 CPUs utilized confirms the single core is saturated with compute for the full duration. Next, we used *perf record* in order to perform a call stack analysis of where the program was spending its CPU time. We found that **99.4%** of runtime is spent in linear matrix multiplications, with **70.1%** occurring in the MLP and **28.3%** in attention (note the program spends around 0.4% performing rmnsnorm, Softmax, and embedding operations). While attention is often considered the bottleneck in Transformer models, this aligns with our earlier complexity analysis, as the approximately constant sequence length causes the d^2 term to dominate, making the MLP the primary contributor to runtime.

Based on the serial workload analysis, our parallelization strategy focuses on exposing more independent work to the GPU. The original *microgpt* structure processes one name at a time and then one token at a time within that name. This creates very little parallel work per operation, especially because the names are short. To increase available parallelism, we batch multiple names together so each kernel operates over many sequences and token positions at once, as shown in Algorithm 2. We denote the batch size as $B = |\mathcal{B}|$ and represent each batch as a tensor of shape $(|\mathcal{B}|, T, d)$. To reduce CUDA kernel complexity, we preprocess the names into batches where all sequences in a batch have the same length.

Algorithm 2 High-Level Batched Forward Pass

loss $\leftarrow 0$
batches $\leftarrow$ CreateBatches($\mathcal{D}$)
for each batch $\mathcal{B} \in$ batches **do**
  $X \leftarrow$ TokenEmbedding($\mathcal{B}$) + PositionalEmbedding($0{:}T-1$)
  $X \leftarrow$ RMSNorm(X)
  for $\ell = 1$ to L **do**
    $X \leftarrow X +$ Attention(RMSNorm(X)) ▷ independent per sequence
    $X \leftarrow X +$ MLP(RMSNorm(X))
  end for
  $Y \leftarrow$ Linear(X)
  loss $\leftarrow$ loss + CrossEntropy(Y, Targets($\mathcal{B}$))
end for
loss $\leftarrow$ loss/total_tokens

Our parallelization strategy focuses on exposing parallelism at the level of individual tensor elements and matrix tiles, rather than at the level of tokens or sequences. This allows each CUDA kernel to operate over $|B| \cdot T \cdot d$ elements simultaneously, significantly increasing available work per launch compared to the serial implementation.

We achieve this by decomposing the forward pass into a sequence of CUDA kernels, each operating at a finer granularity over the batched tensor. Rather than treating attention and MLP as monolithic operations, we explicitly map each sub-operation (RMSNorm, linear projections, attention, and residual additions) to separate GPU kernels. This decomposition is shown in Algorithm 3, where each step corresponds directly to a kernel launch in our implementation.

Algorithm 3 Layer-by-Layer GPU Forward Pass

for $\ell = 1$ to L **do**
  $X_{\text{norm}} \leftarrow \text{RMSNorm}(X)$ ▷ **Attention Block**
  $Q \leftarrow \text{Linear}(X_{\text{norm}}, W_Q^{(\ell)})$
  $K^{(\ell)} \leftarrow \text{Linear}(X_{\text{norm}}, W_K^{(\ell)})$
  $V^{(\ell)} \leftarrow \text{Linear}(X_{\text{norm}}, W_V^{(\ell)})$
  $A \leftarrow \text{SelfAttention}(Q, K^{(\ell)}, V^{(\ell)})$
  $A_{\text{proj}} \leftarrow \text{Linear}(A, W_O^{(\ell)})$
  $X_{\text{mid}} \leftarrow X + A_{\text{proj}}$
  $X_{\text{norm2}} \leftarrow \text{RMSNorm}(X_{\text{mid}})$ ▷ **MLP Block**
  $H \leftarrow \text{Linear}(X_{\text{norm2}}, W_1^{(\ell)})$
  $H \leftarrow \text{ReLU}(H)$
  $H_{\text{proj}} \leftarrow \text{Linear}(H, W_2^{(\ell)})$
  $X \leftarrow X_{\text{mid}} + H_{\text{proj}}$
end for

Each operation in Algorithm 3 is implemented as a dedicated CUDA kernel, as summarized in Table 1.

Operation	**Thread Granularity**	**Description**
Embedding + Positional	element (B, T, d)	each thread loads one embedding value
RMSNorm	token (B, T)	each thread normalizes one token by looping over its d values
SelfAttention	element (B, T, d)	one thread computes one weighted scalar within its attn head
Linear (Q, K, V, MLP)	element $(T, \text{out\_dim})$	tiled matix mul, one thread owns one output cell
ReLU	element (B, T, d)	independent element-wise activation
Vector Add	element (B, T, d)	independent element-wise vector addition
CrossEntropy	single thread	one thread loops over all (B, T, vocab) to compute cross-entropy

Table 1: Kernel decomposition with thread granularity and execution details.

From a systems perspective, the full model is transferred to the GPU once at initialization, while a reusable workspace is allocated once per batch. This avoids repeated host-device transfers and ensures that all intermediate activations remain resident on the GPU throughout the forward pass.

With this strategy in place, we will next evaluate how effectively the batched GPU implementation reduces wall-clock time compared to serial and PyTorch baselines.

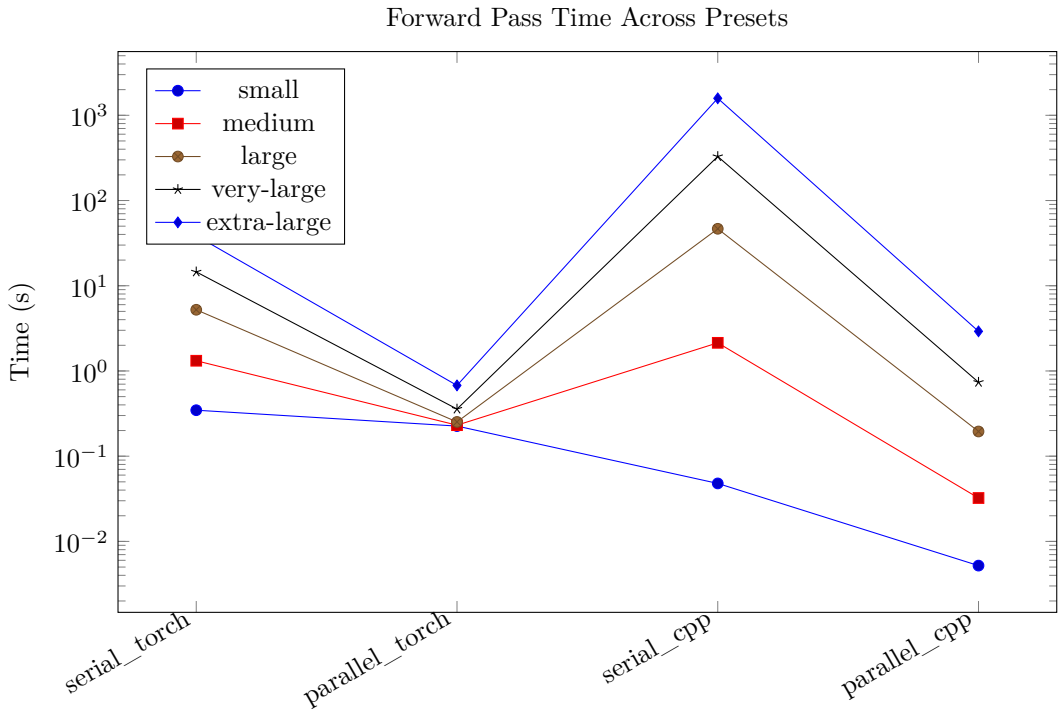
4 Performance Results with Graphs

The following sections present performance graphs across the evaluated model presets. We compare PyTorch, serial C++, and our custom parallel CUDA implementation to analyze forward-pass runtime across configurations. For each preset, we include a plot highlighting serial C++ performance as a baseline and the resulting speedup obtained through parallelization.

Preset	**n_layer**	**n_embd**	**block_size**	**n_head**	**steps**
small	1	64	128	4	200
medium	2	128	128	8	1000
large	4	256	256	8	2500
very-large	6	384	512	12	5000
extra-large	8	512	512	16	10000

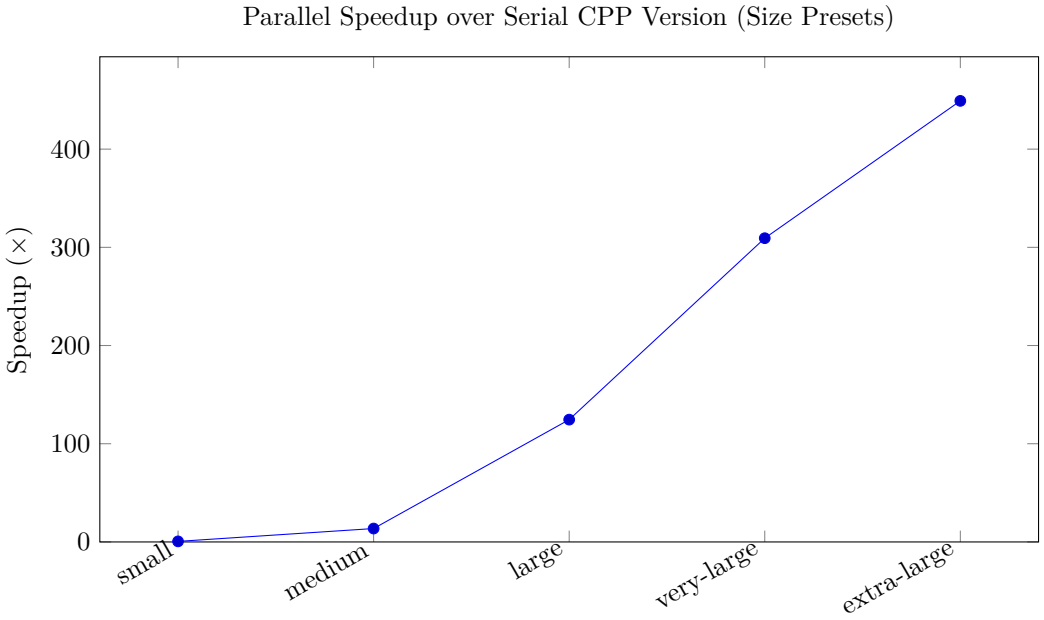
Table 2: Model and Name Size Configurations

The first set of presets range in both model size and the number of names that we are performing the forward pass on. The aim of these presets was to determine what the forward pass time looked like across the various implementations as we scaled every parameter up. As you can see in graph 4, our implementation surpassed the serial C++ version (our baseline) on every preset. Additionally, we outperformed the PyTorch implementations for smaller dataset sizes.



The following figure illustrates the speedup achieved by our parallel implementation relative to the serial C++ baseline. As expected, the parallel version outperforms the serial implementation across all preset sizes. The trend also highlights that the performance advantage grows with increasing workload, since larger presets provide more

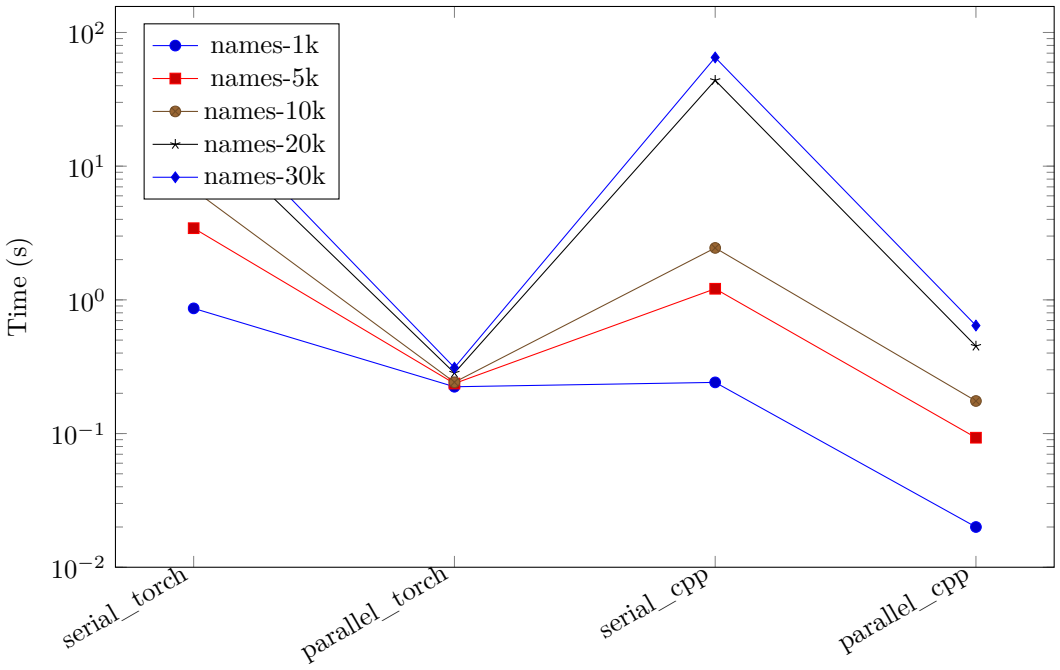
opportunities to amortize kernel launch overhead and better utilize GPU parallelism. Notably, for the largest configurations, the forward pass achieves speedups exceeding 300×, indicating that the implementation scales particularly well under high compute intensity where the GPU is able to remain fully saturated.



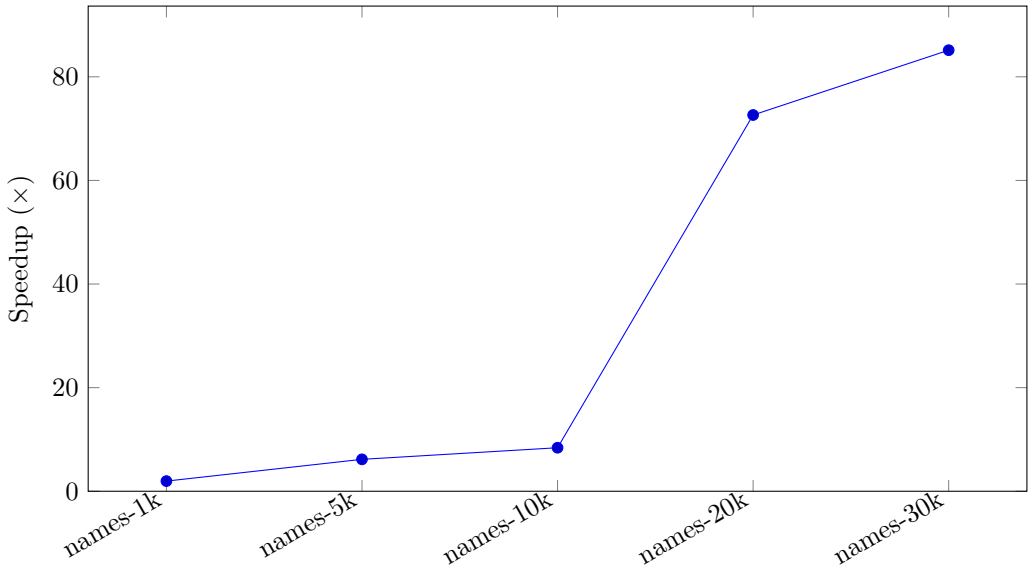
Preset	n_layer	n_embd	block_size	n_head	steps
names-1k	1	64	512	4	1000
names-5k	1	64	512	4	5000
names-10k	1	64	512	4	10000
names-20k	2	128	512	8	20000
names-30k	2	128	512	8	30000

Table 3: Name Size Configurations

Forward Pass Time Across Names Presets

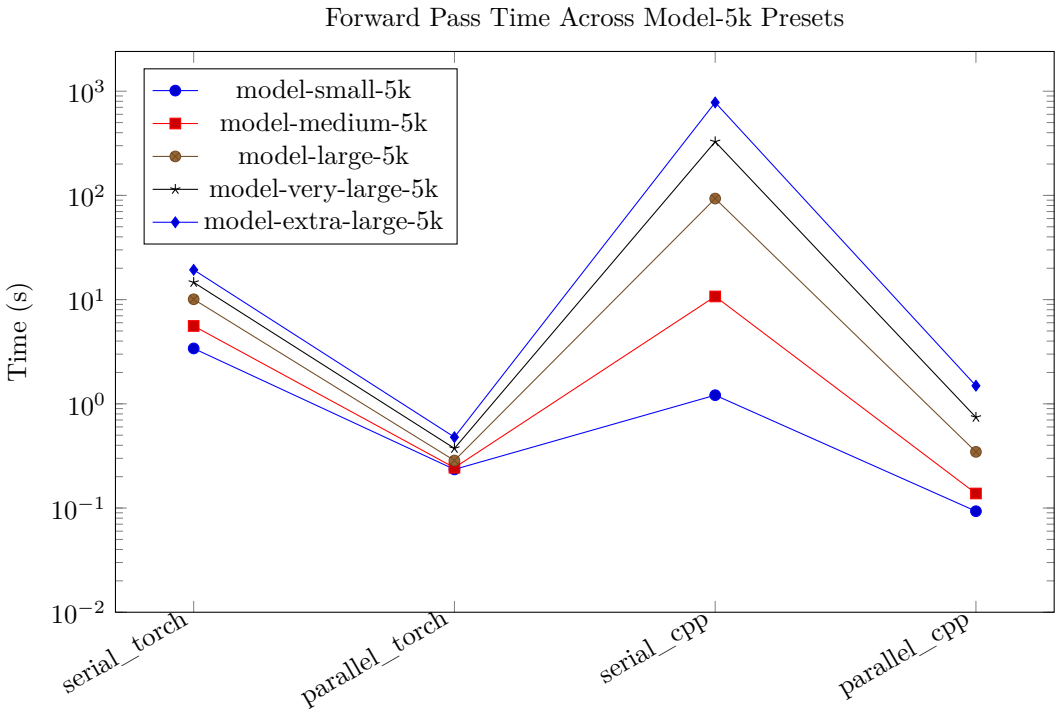


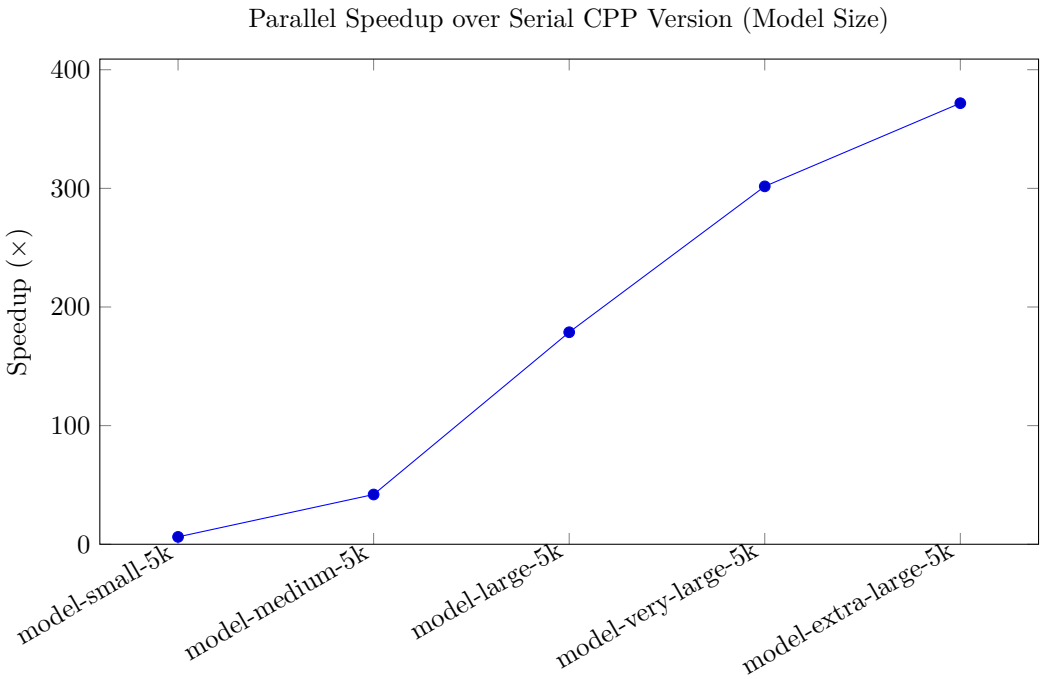
Parallel Speedup over Serial CPP Version (Names Size)



Preset	n_layer	n_embd	block_size	n_head	steps
model-small-5k	1	64	512	4	5000
model-medium-5k	2	128	512	8	5000
model-large-5k	4	256	512	8	5000
model-very-large-5k	6	384	512	12	5000
model-extra-large-5k	8	512	512	16	5000

Table 4: Model Size Configurations





5 Bottleneck Discussion

Placeholder text.

6 Lessons Learned

Placeholder text.

References

[1] Andrej Karpathy. microgpt, February 2026. Blog post. https://karpathy.github.io/2026/02/12/microgpt/.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, February 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.